Proyecto Final — Clasificador de Landmarks con CNN

Resumen para presentación docente

Autor: Klaus Uribe — Maestría en Ciencia de Datos e IA Aplicada, Módulo 6 (Redes Neuronales)
Fecha: 22 de abril de 2026 **Stack:** Python 3.11 · PyTorch 2.5.1 + CUDA 12.1 · NVIDIA RTX 2050 (4 GB VRAM)

🎯 Objetivo

Construir un clasificador multiclase de **50 landmarks** aplicando y comparando dos enfoques:

1. Una CNN **diseñada desde cero** (umbral mínimo: 40% test accuracy).
2. **Transfer Learning** con ResNet50 preentrenada en ImageNet (umbral: 70%; bono: +75%).

Adicionalmente, una aplicación de predicción con interfaz Gradio (bonificación +1 punto).

Fase 0 — Setup del entorno

Entorno aislado y reproducible. Se usó **Python 3.11.9** específicamente (el default del sistema es 3.14, incompatible con las ruedas actuales de PyTorch). `venv/` dedicado, `requirements.txt` con el index oficial de PyTorch para CUDA 12.1. GPU verificada mediante `test_gpu.py` (PyTorch detecta RTX 2050, 4.29 GB VRAM, compute capability 8.6). Estructura de carpetas estándar: `data/`, `notebooks/`, `src/`, `models/`, `images_test/`, `report/`.

Fase 1 — Exploración y preprocesamiento

Dataset (subconjunto Google Landmarks v2):

- 50 clases · **4 996** imágenes de train (≈100/clase) · **1 250** de test (25/clase exacto)
- Balance confirmado: desviación estándar de 0.56 imágenes/clase → dataset balanceado, **no hacen falta `class_weights` ni oversampling**.

Pipeline de transformaciones, diferenciado por split:

Split	Transformaciones
Train	Resize(256) → RandomResizedCrop(224, scale 0.8-1.0) → HorizontalFlip → Rotation(15°) → ColorJitter(0.2) → ToTensor → Normalize(ImageNet)
Val / Test	Resize(256) → CenterCrop(224) → ToTensor → Normalize(ImageNet)

Justificaciones clave:

- *Augmentation solo en train* para combatir overfitting con pocas imágenes/clase sin corromper la evaluación.
- *Normalización con media/desvío de ImageNet* para mantener compatibilidad con ResNet50 (Fase 3).
- *Resize 256 + CenterCrop 224* preserva aspect ratio — pipeline estándar de ImageNet.

Splits finales: Train 4 247 (85%) · Val 749 (15%) · Test 1 250 (intacto).

Visualizaciones generadas y embebidas en `report/`: distribución de clases, muestras del dataset, batch post-augmentation.

Fase 2 — CNN desde cero

Arquitectura `LandmarkCNN`: 4 bloques Conv3×3 → BatchNorm → ReLU → MaxPool con progresión de canales 3 → 32 → 64 → 128 → 256 · GlobalAveragePool · FC(256→512) con Dropout(0.5) · FC(512→50). **546 610 parámetros** — compacta frente a los 23.6M de ResNet50.

Decisiones de diseño y sus porqués:

Decisión	Justificación
4 bloques conv (mín. 3 del proyecto)	Balance capacidad/riesgo de overfitting con ~100 imgs/clase
BatchNorm después de cada Conv	Estabiliza entrenamiento, reduce sensibilidad al LR
GlobalAvgPool (no Flatten)	Reduce ~50M a ~130K parámetros en la transición a FC
Dropout 0.5 (agresivo)	Contrarresta overfitting inherente a datasets chicos
Adam lr=1e-3, weight_decay=1e-4	Default práctico moderno para visión

Proceso de entrenamiento en dos corridas

Corrida inicial (30 épocas): 16.6 min · Val loss cayó monotónicamente 3.63 → 2.44 · Val acc 11.5% → 37.5%. **Test acc: 39.20%** — 0.80 pts bajo el umbral. **Diagnóstico:** subentrenamiento, *no* overfitting. La curva val seguía mejorando en la última época.

Continuación desde checkpoint (hasta 15 épocas más, early stopping manual):

- Se cargó `cnn_scratch_best.pt`, mismo optimizador (Adam lr=1e-3, wd=1e-4) y scheduler (`ReduceLROnPlateau`).
- Regla de detención: 3 subidas consecutivas de val_loss.
- Se ejecutaron **11 épocas** antes del disparo del early stop (épocas globales 31-41).
- Mejor modelo: época global 38 (val_loss 2.36, val_acc 40.05%).
- **Test acc final: 41.92%** (+2.72 pts) · ✅ **APROBADO**.

Tiempo total Fase 2: 22.5 min · **41 épocas efectivas**.

Fase 3 — Transfer Learning con ResNet50

Arquitectura: ResNet50 preentrenada en ImageNet (`IMAGENET1K_V2`), 23.6M parámetros. Cabeza clasificadora original reemplazada por `Dropout(0.4) → Linear(2048 → 50)`.

Por qué ResNet50 (y no VGG16, ResNet18, EfficientNet): Balance óptimo para el hardware y dataset:

- **Skip connections** permiten propagar gradientes en redes profundas sin vanishing.
- 25M params cabe cómodo en 4 GB VRAM con batch=16.
- Pesos IMAGENET1K_V2 son los de mayor calidad disponibles en torchvision.
- Ampliamente documentada, reproducible.

Entrenamiento en dos sub-fases

Sub-fase	Estrategia	Params entrenables	Épocas	LR	Tiempo	Mejor val_loss
A	Backbone congelado (solo cabeza)	102 450 (0.43%)	15	1e-3	10.3 min	0.8354 (ép. 15)
B	Fine-tuning de `layer4`	15 067 186 (63.82%)	10	1e-4	8.1 min	**0.7227 (ép. 1)**

Observación técnica: el mejor val_loss del fine-tuning apareció en la primera época de la sub-fase B. `layer4` adaptó sus features al dominio de landmarks casi instantáneamente. En las 9 épocas restantes apareció un leve overfitting (train acc 98.7% vs val estable), pero el mejor checkpoint ya estaba guardado.

Resultado: Test accuracy 82.72% · ✅ **APROBADO** + bono +75% alcanzado.

- +12.72 pts sobre el umbral mínimo (70%)
- +7.72 pts sobre el umbral del bono (75%)

Tiempo total Fase 3: 18.4 min.

Fase 4 — Aplicación de predicción + Gradio

Módulo `src/predictor.py` con tres funciones públicas:

- `get_inference_transform()` — pipeline determinístico (consistente con val/test).
- `load_model(path)` — carga modelo TorchScript (portable, sin dependencia de la clase Python original).
- `predict_landmarks(img, model, classes, k=3)` — retorna top-k `(clase, probabilidad)` aplicando Softmax sobre logits.

Validación con imágenes propias

Imagen	Top-1	Confianza	Top-1 correcto
Central Park	`15.Central_Park`	96.5%	✅
Golden Gate Bridge	`09.Golden_Gate_Bridge`	78.5%	✅
Petronas Towers	`32.Hanging_Temple`	28.5%	❌ (correcta en top-3, 9.1%)
Torre Eiffel	`16.Eiffel_Tower`	87.6%	✅

Resultados: Top-1 accuracy **3/4 (75%)** · Top-3 accuracy **4/4 (100%)**.

Análisis: el caso Petronas es diagnóstico — el modelo dispersa probabilidad sobre clases de arquitectura asiática (Hanging Temple, Changdeokgung), lo que sugiere que el ejemplar subido tiene un encuadre/iluminación no prototípicos en el training set. Que la clase correcta aún aparezca en top-3 muestra que el espacio de features es coherente.

Interfaz Gradio: construida con `gr.Image → gr.Label(top-5)`. Lanzable localmente con `demo.launch(share=False, inbrowser=True)`. El lanzamiento está protegido por un flag (`LAUNCH_GRADIO`) para que la ejecución batch de los notebooks no quede bloqueada en el servidor interactivo.

📊 Comparación consolidada de modelos

Modelo	Parámetros	Épocas	Tiempo	Test Accuracy	Estado
CNN from Scratch	546 K	41 (30 + 11)	22.5 min	**41.92%**	✅ ≥ 40%
Transfer Learning (ResNet50)	23.6 M	25 (15 + 10)	18.4 min	**82.72%**	✅ ≥ 70% + bono

Insight central: Transfer Learning logra **+40.8 pts** de test accuracy usando ~43× más parámetros pero aprovechando el preentrenamiento masivo de ImageNet (1.2M imágenes). El tiempo de

cómputo adicional es mínimo (~2.3× más tiempo efectivo), con mejor retorno por minuto de entrenamiento.

🔬 Hallazgos técnicos relevantes

1. **Con augmentation fuerte, `train_acc < val_acc` es normal.** Durante toda la Fase 2 la curva de train quedó por debajo de val — fenómeno esperable porque train aplica augmentation y val no. La métrica adecuada para diagnosticar subentrenamiento vs overfitting fue la trayectoria de **val_loss**, no la comparación visual train/val.
2. **El early stopping manual confirmó el techo de la CNN propia.** Las 3 subidas consecutivas de val_loss en ép. 39→40→41 dispararon la detención exactamente donde el modelo había dejado de aprender.
3. **Fine-tuning ultra-rápido en Fase 3B.** Que el mejor val_loss apareciera en la primera época del fine-tuning es evidencia concreta del valor del preentrenamiento: `layer4` sólo necesitó *un pase* sobre el dataset de landmarks para especializarse.
4. **Top-k vs top-1 revela la calidad del espacio de features.** Que un modelo alcance 100% top-3 con 75% top-1 en una muestra pequeña sugiere que las features aprendidas son correctas pero la separabilidad entre clases visualmente similares (Petronas vs templos orientales) tiene margen de mejora.

⚠️ Limitaciones y mejoras futuras

- **Dataset pequeño** (100 imgs/clase) es el techo principal de la CNN custom.
- **VRAM 4 GB** obligó batch=16 en ResNet50; con GPU más grande se podría usar batch 32-64 y obtener gradientes más estables.
- **Mejoras plausibles** en orden de costo/beneficio: test-time augmentation · ensemble CNN scratch + ResNet50 · cosine annealing con warmup · augmentation moderna (MixUp, CutMix) · fine-tuning progresivo por capas.

📦 Entregables

- **5 notebooks ejecutados** con outputs inline (gráficos y tablas embebidas): `00_exploracion_y_preproceso` · `01_cnn_from_scratch` · `01b_cnn_scratch_continuation` · `02_transfer_learning` · `03_app_prediccion`
- **5 modelos guardados** (`models/`): checkpoints `.pt` y versiones TorchScript exportadas de ambas arquitecturas.
- **7 gráficos** (`report/`): distribución de clases, muestras, batch con augmentation, curvas CNN scratch original y continuación, curvas Transfer Learning, comparación final, predicciones sobre imágenes propias.

- **Código modular** (`src/`): `data.py` , `model.py` , `train.py` , `predictor.py` — reutilizable fuera de los notebooks.
- **Reproducibilidad:** seed global = 42 · `requirements.txt` · `SETUP.md` · `test_gpu.py` .